endava

UNIVERSITATEA DIN BUCUREȘTI
VIRTUTE ET SAPIENTIA

PROGRAMARE ORIENTATA PE OBIECTE II

CURS 2

AGENDA CURS 2

1. Arrays
2. Clase si obiecte
3. Atribute
4. Zone de memorie constante
5. Zone de memorie de clasa
6. Constructori
7. Comportamente
8. Metode de clasa
9. Ciclul de viata al unui obiect
10. Modificatori de acces
11. Pachete

ARRAYS

- Structura de date cu dimensiune fixa
- Stocheaza elemente cu acelasi tip de date
- Declarare:

```
int[] numbers;  // Preferred syntax
int numbers2[]; // Also valid
```

- Initializare:

```
numbers = new int[5]; // Array of size 5 (default values: 0 for int, null for objects)
```

- Declarare si initializare:

```
int[] numbers = {10, 20, 30, 40, 50}; // Array with values
String[] names = new String[]{"Alice", "Bob", "Charlie"};
```

ARRAYS

- Accesarea unui element dintr-un array:

```
System.out.println(numbers[0]); // Access first element
numbers[1] = 25;  // Modify second element
```

- Parcurgerea unui array:
  - for

```
for (int i = 0; i < numbers.length; i++) {
    System.out.println(numbers[i]);
}
```

  - enhanced for (for each)

```
for (int num : numbers) {
    System.out.println(num);
}
```


endava

ARRAYS

- Array multidimensional:

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};

// Accessing elements
System.out.println(matrix[1][2]); // Prints 6
```

ARRAYS

Aspect	Avantaje ✅	Dezavantaje ❌
Performanta	Acces rapid la elemente (`O(1)`) datorita indexarii directe	Inserarea si stergerea sunt lente (`O(n)`) deoarece necesita mutarea elementelor
Memorie	Eficient pentru date fixe, utilizare optimizata a memoriei	Dimensiune fixa → poate duce la risipa de memorie sau la realocari costisitoare
Simplitate	Usor de utilizat si implementat	Necesita gestionare manuala pentru redimensionare
Flexibilitate	Potrivit pentru date de lungime fixa	Nu poate fi redimensionat dinamic (spre deosebire de `ArrayList`)
Functionalitati	Stocare simpla si rapida pentru tipuri de date omogene	Nu ofera metode incorporate pentru adaugare, stergere sau cautare avansata (`ArrayList` este mai flexibil)

CLASE SI OBIECTE

- Clasa:
  - blueprint / template pentru crearea de obiecte
  - se defineste cu **class** keyword

- Obiect:
  - o instanta concreta a unei clase
  - se defineste cu **new** keyword
  - pe baza unei clase se pot crea oricat de multe obiecte

Blueprint

Houses built according to the blueprint

ATRIBUTE

- zone de memorie care stocheaza caracteristici ale unui obiect
- attributes = fields = instance variables
- fiecare obiect are propria copie a atributelor

```
public class Car {
    // Atribute (Variabile de instanta)
    String brand;
    int speed;
}
```

```
Car car1 = new Car();
car1.brand = "Toyota";

Car car2 = new Car();
car2.brand = "Ford";  // Valoare diferita pentru un alt obiect
```


endava

ATRIBUTE

- in lipsa unui specificari explicite, atributele sunt variabile si apartin unei instante
- daca se doreste ca toate obiectele create pe baza unei clase sa aiba aceeasi valoare pentru un atribut, atributul respectiv poate fi marcat ca o zona de memorie constanta, cu modificatorul **final**

```
public class Car {
    final String model = "Tesla"; // Nu poate fi modificat
}
```


endava

ZONE DE MEMORIE DE CLASA

- sunt accesibile tuturor obiectelor create pe baza unei clase
- pot fi accesate prin numele clasei (recomandat), dar si prin referinte de obiecte (nerecomandat)
- Use cases:
  - partajarea unei resurse de catre toate obiectele unei clase, fara a o duplica
  - setari / configurari (API URL, connection URL)

```
public class Car {
    static int totalCars = 0; // Comun pentru toate obiectele
}
```

```
Car.totalCars = 5;  // Se acceseaza prin numele clasei
```


endava

CONSTRUCTORI

- metoda speciala a unei clase care este apelata automat atunci cand un obiect este creat
- sunt folositi pentru a initializa atributele unui obiect
- au acelasi nume precum clasa
- nu au tip returnat (nici macar void)

endava

CONSTRUCTORI

- **Constructor default** (fara parametrii): este definit explicit sau este adaugat de catre compilator, daca niciun constructor explicit nu este definit

```java
public class Car {
    String brand;
    int speed;

    // Constructor implicit
    public Car() {
        brand = "Default";
        speed = 0;
    }
}
```

```java
Car myCar = new Car();
System.out.println(myCar.brand); // Output: Default
```


endava

CONSTRUCTORI

- **Constructor cu parametrii**

```java
public class Car {
    String brand;
    int speed;

    // Constructor parametrizat
    public Car(String brand, int speed) {
        this.brand = brand;
        this.speed = speed;
    }
}
```

```java
Car myCar = new Car("Toyota", 60);
System.out.println(myCar.brand); // Output: Toyota
```


endava

CONSTRUCTORI

- putem defini mai multi constructori in aceeasi clasa, prin mecanismul de **supraincarcare**
- pentru a beneficia de supraincarcare, lista de parametrii a doi constructori trebuie sa fie diferita prin cel putin unul dintre criteriile:
  - numar de parametrii
  - tipul de data al parametrilor
  - ordinea tipului de data al parametrilor

```java
public class Car {
    String brand;
    int speed;

    // Constructor implicit
    public Car() {
        this.brand = "Default";
        this.speed = 0;
    }

    // Constructor parametrizat
    public Car(String brand, int speed) {
        this.brand = brand;
        this.speed = speed;
    }
}
```

CONSTRUCTORI

- un constructor poate apela un alt constructor al aceleiasi clase cu **this()**
- un constructor poate accesa instanta obiectului care se construieste prin **this**

```java
public class Car {
    String brand;
    int speed;

    // Constructor implicit
    public Car() {
        this("Default", 0); // Apel catre constructorul parametrizat
    }

    // Constructor parametrizat
    public Car(String brand, int speed) {
        this.brand = brand;
        this.speed = speed;
    }
}
```


endava

ATRIBUTE

Cum si unde putem da valori atributelor?

1. la declarare:
   - valoarea este atribuita direct in momentul declararii atributului
   - se foloseste cand vrei ca toate obiectele sa aiba o valoare implicita pentru un atribut

```
public class Car {
    String brand = "Toyota";
    int speed = 50;
}
```

2. prin constructor
   - permite initializarea atributelor in momentul crearii obiectului
   - permite initializarea atributelor cu valori diferite pentru fiecare obiect

```
public class Car {
    String brand;
    int speed;

    // Constructor
    public Car(String brand, int speed) {
        this.brand = brand;
        this.speed = speed;
    }
}
```


endava

ATRIBUTE

3. prin bloc de initializare
   - se executa inaintea constructorului
   - se foloseste cand ai nevoie de **initializare comuna** pentru toti constructorii clasei

```
public class Car {
    String brand;
    int speed;

    // Bloc de initializare
    {
        brand = "Mercedes";
        speed = 100;
    }
}
```


endava

COMPORTAMENTE

- comportamentele unei clase sunt definite prin **metode**, care reprezinta actiunile pe care obiectele clasei le pot efectua
- permite obiectelor sa isi modifice starea si sa interactioneze cu alte obiecte
- pot accesa atributele obiectului care se comporta, folosind **this**
- this poate fi accesat in orice metoda non-statica dintr-o clasa (constructor sau comportament)
- comportamentele pot fi apelate doar folosind referinta unui obiect

```java
public class Car {
    String brand;
    int speed;

    // Constructor
    public Car(String brand, int speed) {
        this.brand = brand;
        this.speed = speed;
    }

    // Metoda pentru accelerare
    public void accelerate() {
        speed += 10;
        System.out.println(brand + " accelereaza la " + speed + " km/h.");
    }
}
```

```java
Car myCar = new Car("Toyota", 50);
myCar.accelerate(); // Output: Toyota accelereaza la 60 km/h.
```

METODE DE CLASA

- sunt metode care apartin **clasei**, nu unui obiect specific
- nu pot accesa atributele unui obiect si nu pot apela comportamentele unui obiect (nu au access la **this**)
- se folosesc pentru operatii care nu depind de atributele unui obiect
- se definesc folosind **static** keyword
- pot fi apelate folosind numele clasei in care au fost definite

```java
public class MathUtils {
    public static int square(int number) {
        return number * number;
    }
}
```

```java
int result = MathUtils.square(5);
System.out.println(result); // Output: 25
```


endava

CICLUL DE VIATA AL UNUI OBIECT

endava

CICLUL DE VIATA AL UNUI OBIECT

Etapa	Descriere	Exemplu	Ce se intampla in memorie?	Observatii
1. Declarare	Se creeaza o referinta catre un obiect, dar obiectul nu este inca alocat in memorie.	`Car myCar;`	Doar o referinta este creata in **Stack Memory**, dar nu exista inca un obiect in **Heap Memory**.	Daca incerci sa accesezi obiectul inainte de alocare, vei obtine o eroare `NullPointerException`.
2. Creare (Instantiere)	Obiectul este creat in memorie folosind `new`.	`myCar = new Car("Toyota", 50);`	Un nou obiect este creat in **Heap Memory**, iar referinta catre acesta este stocata in **Stack Memory**.	Constructorul este apelat automat si initializeaza variabilele obiectului.

endava

CICLUL DE VIATA AL UNUI OBIECT

3. Utilizare	Se folosesc metodele si variabilele obiectului.	`myCar.accelerate();`	Obiectul ramane activ in **Heap Memory** si poate fi utilizat prin referinta.	Obiectul poate fi folosit atata timp cat exista cel putin o referinta activa catre el.
4. Dezalocare (Referinta devine `null`)	Referinta la obiect este eliminata, dar obiectul inca exista in memorie.	`myCar = null;`	Referinta este eliminata din **Stack Memory**, dar obiectul inca ocupa loc in **Heap Memory**.	Daca nu mai exista nicio referinta activa, obiectul devine **eligibil** pentru Garbage Collection.
5. Distrugere (Garbage Collection)	Java detecteaza ca obiectul nu mai este utilizat si il sterge automat.	`System.gc();` *(sugereaza rularea GC)*	Garbage Collector-ul elibereaza memoria ocupata in **Heap Memory**.	Garbage Collector-ul ruleaza automat, dar momentul exact nu este garantat.

endava

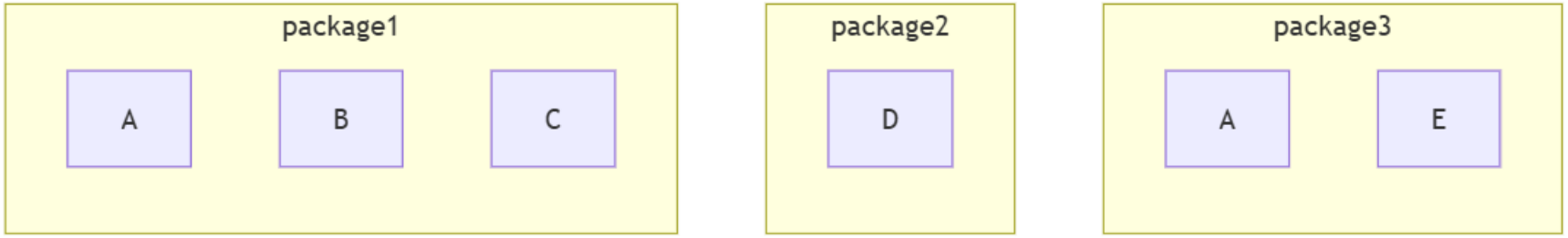
MODIFICATORI DE ACCES

Modificator	Accesibil in Aceeasi Clasa	In Acelasi Pachet	Subclasa (Alt Pachet)	Alte Clase
`public`	✅	✅	✅	✅
`protected`	✅	✅	✅	❌
default (fara modificator)	✅	✅	❌	❌
`private`	✅	❌	❌	❌

endava

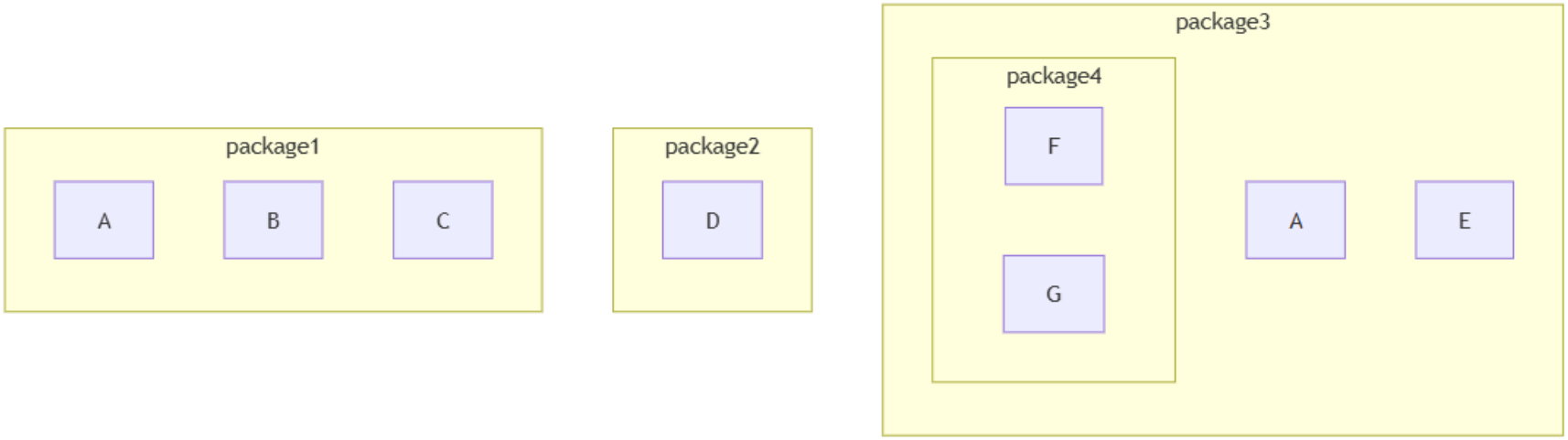
PACHETE

- un pachet este un grup de clase organizate intr-o structura logica
- pachetele ajuta la organizarea codului, evitarea conflictelor de nume si controlul accesului
- un pachet se defineste cu **package**
- pentru a folosi intr-o clasa A din pachetul package1 o alta clasa D din pachetul package2:
  - clasa A trebuie sa importe clasa D
  - clasa A foloseste fully qualified name al clasei D, care include numele pachetului din care face parte (package2.D). Aceasta metoda evita conflictele de nume atunci cand exista clase cu acelasi nume in pachete diferite.
- importul de clase se face cu **import**



PACHETE

- un pachet nedeclarat explicit este default package. Clasele din default package nu pot fi importate in alte pachete
- un pachet poate contine **subpackages** pentru organizare mai buna (de exemplu, package4 este un subpackage pentru package3)



Q&A

THANK YOU

DANIELA SPILCĂ